

Editorial Pedagógico: DNA

Por: WOK

Data: Maio 2026

Skills: Programação Dinâmica, Strings

▷ Objetivo Pedagógico

Objetivos Pedagógicos:

- Desenvolver proficiência na clássica técnica de Programação Dinâmica bi-dimensional (LCS - Longest Common Subsequence).
- Entender como adicionar restrições condicionais baseadas em estados contínuos (tamanho mínimo de bloco).
- Aprender a otimizar a transição de estado da DP de $O(N^3)$ para $O(N^2)$ usando vetores auxiliares.

Disciplinas Relacionadas

Disciplina	Tópico Aplicado
Algoritmos e Estruturas de Dados II	Programação Dinâmica e Processamento de Strings (Bioinformática).
Análise de Complexidade	Redução do custo da função de transição de $O(N)$ para $O(1)$ constante em cada célula.
Lógica Matemática	Avaliação de subproblemas ótimos e independência de blocos de prefixo.

Programação Dinâmica

DP 2D

Bioinformática

Blocos Contíguos

Otimização $O(N^2)$

Editorial Completo

Neste problema, o desafio central não é apenas encontrar a maior subsequência em comum (como no clássico LCS), mas sim impor que as letras escolhidas devem compor **segmentos contínuos de no mínimo tamanho K** .

A abordagem inocente seria, a cada célula (i, j) de uma matriz padrão, varrer um contador c de K até o número máximo de combinações diretas (gerando complexidade cúbica $O(N^3)$). Em competições e sistemas reais de bioinformática, isso levará a um Time Limit Exceeded (TLE) para strings medianas.

Alerta de Eficiência: Se você tem três laços `for` aninhados iterando sobre os limites das strings, seu código é $O(N^3)$ e não passará. Precisamos eliminar o terceiro laço mantendo o "status" do maior bloco no estado da DP.

Modelando o Problema

Utilizaremos três matrizes (ou uma matriz tridimensional simplificada) para armazenar subproblemas:

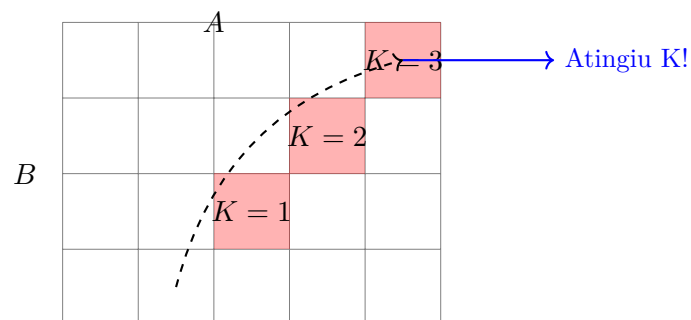
1. **match[i][j]**: O comprimento exato do sufixo contínuo terminando em $A[i-1]$ e $B[j-1]$. Se os caracteres são iguais, soma 1 à diagonal superior esquerda. Caso contrário, zera.
2. **dp[i][j]**: O melhor resultado que pode ser obtido até os prefixos i e j .
3. **f[i][j]**: A melhor subsequência que **obrigatoriamente** utiliza o caractere atual (i, j) como fim de um bloco estendido ou recém-formado com tamanho $\geq K$.

Quando **match[i][j]** atinge $\geq K$, abre-se duas portas: - Começar um novo bloco isolado que consome os últimos K caracteres: $dp[i-K][j-K] + K$. - Estender um bloco que **já vinha** crescendo do índice anterior: $f[i-1][j-1] + 1$.

Fórmula Direta de Transição: Seja $A[i-1] == B[j-1]$ e $match[i][j] \geq K$:

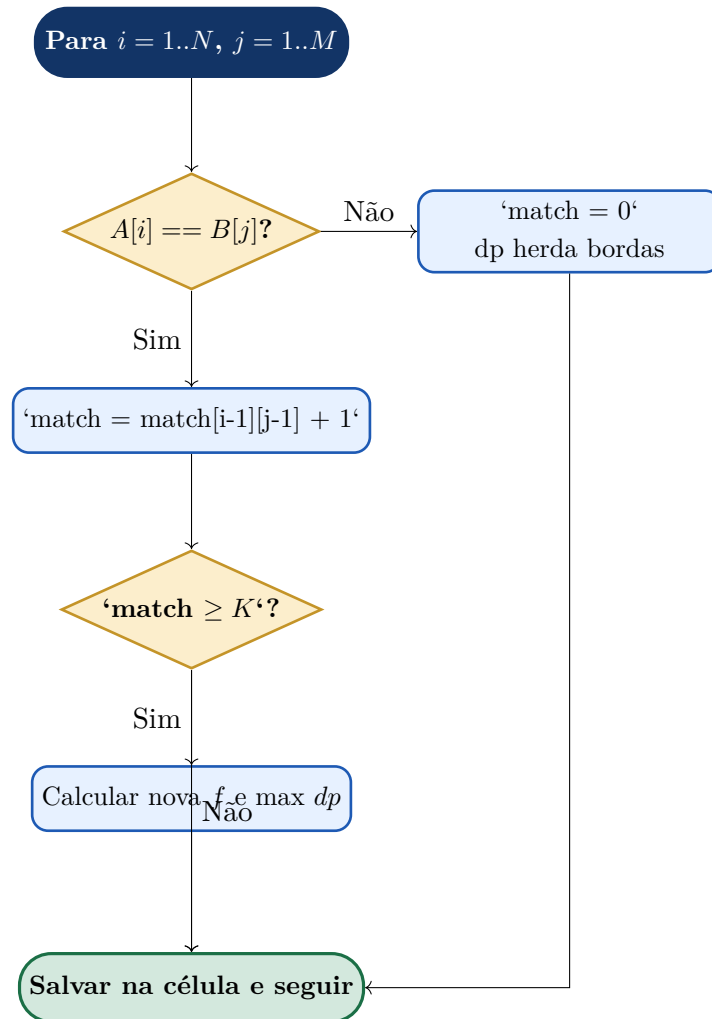
$$f[i][j] = \max(dp[i-K][j-K] + K, f[i-1][j-1] + 1)$$

Em seguida: $dp[i][j] = \max(dp[i-1][j], dp[i][j-1], f[i][j])$.



Fluxograma da Solução

Abaixo ilustra-se as bifurcações de raciocínio a cada avanço no grid da DP.



Análise de Complexidade

Fase	Tempo	Espaço	Observação
Alocação das Matrizes	$O(N^2)$	$O(N^2)$	Três matrizes de tamanho $N \times M$. Caso limite, pode-se economizar memória guardando só 2 linhas.
Travessia da Programação Dinâmica	$O(N^2)$	$O(1)$	Iteração dupla clássica onde cada par avaliado processa um limite fixo de comparações independentes do tamanho da string.
Total	$O(N^2)$	$O(N^2)$	Uma redução colossal frente ao $O(N^3)$, validando perfeitamente a barreira de 1 segundo de tempo de CPU.

Tutorial Recomendado: Para um profundo aprofundamento na família de problemas *Longest Common Subsequence* e como aplicar transições avançadas envolvendo blocos lógicos contíguos (muito comum no ramo da Bioinformática, que estuda pareamento de sequências genômicas), consulte o clássico "Introduction to Algorithms" (Cormen, Leiserson, Rivest, Stein), capítulo 15 sobre Programação Dinâmica. Adicionalmente, treine outras restrições e topologias de grids no *Competitive Programming 3* ou *4*.